

Vahan Lead-Source Analysis

Product Analytics Internship Case Study

By Yamini D

Yaminiduraisamy14@gmail.com

Linkedin: www.linkedin.com/in/yamini-d-126a36290

GitHub: [Yamini1727](https://github.com/Yamini1727)

This document walks through everything I did for this case study — what question I was answering, how I checked the data before trusting it, what I found, and how I built a model on top of it. The full code is in the notebook (Vahan_Case_Study_Analysis.ipynb) and the GitHub repo

1. What This Case Study Is About

Vahan gets candidate leads from a bunch of different sources - referrals, OLX, Khanna's various campaigns, and a few others and calls them to try to get them to complete their first trip (referred to as "FT" throughout, short for First Trip). I was given a dataset of 18,198 leads across 16 different lead sources, uploaded between 18 July and 6 August 2026, and asked to answer three questions:

-
- Which lead sources actually convert best, and by what measure of "best"?
 - Write a query that summarizes the funnel by lead source.
 - Build a model that shows what factors are associated with a lead reaching FT.

I did this in a Jupyter notebook (Python + SQL) so the whole thing is reproducible, and this document is my write-up of the reasoning behind each step.

2. The Data, and Checking It Before I Trusted It

Before I analyzed anything, I checked the data instead of assuming it was clean. Bad inputs make every downstream number wrong.

The dataset has 18,198 rows. Each row is a lead, with a lead source, an upload date, and a set of yes/no flags showing how far that lead got: Attempted (were they called), Connected (did the call go through), Interested, Onboarded, and finally FT.

I ran six checks on the raw data:

- Duplicate rows — none found.
- Flag columns containing something other than 0 or 1 — none found.
- Funnel logic broken (e.g. a lead marked as reaching FT without ever being marked as Attempted) — none found.

- Missing candidate phone numbers — found 1.
- Negative "time to first attempt" values (i.e. the data claiming a lead was called before it was even uploaded, which isn't physically possible) — found 537 rows out of 11,973 attempted leads (4.5%).
- Dates that don't parse — none found.

I did this because a rare-event analysis like this one (FT only happens 54 times in the whole dataset) breaks easily if even a handful of bad rows sneak in, so I wanted to know exactly what was wrong before deciding how to handle it, rather than silently dropping or ignoring anything.

How I fixed what I found

- The 1 row with no candidate phone number — I dropped it. I need a phone number to tell candidates apart later on (Section 4), so a row without one can't be used safely.
- The 537 rows with an impossible negative "time to first attempt" . I did not drop these, I fixed them. This bug was concentrated in two sources (Khanna AI and OLX - Ashwin), and only 1 of the 17 people who eventually reached FT was affected. With so few FT cases to begin with, I didn't want to risk throwing away 4.5% of the data and accidentally losing one of the rare positive cases. I replaced each bad value with the typical (median) time-to-first-attempt for that same lead source, not the overall average - so that a naturally fast-calling source isn't accidentally given a slow source's typical number, or vice versa.
- I also found 116 rows where a candidate was marked "Onboarded" despite never being marked "Interested" - an inconsistency in how the funnel was logged. I didn't need to fix this one because neither of those two columns ends up being used in the model anyway, but it's worth Vahan's data team knowing this logging gap exists.

3. Question 1 — Which 3 cohorts are the best, and on what metric?

Picking the right way to measure "best"

Before ranking anything, I had to decide what "best metric" even means, because different metrics point to different winners. I considered three options and picked the third:

- Raw number of FTs — rejected. This just rewards whichever source has the most volume, not the one that converts efficiently. A source with 5,000 leads and a low conversion rate would always beat a small, excellent source, which doesn't tell Vahan where to spend the next sourcing rupee.
- Interested → FT rate — rejected. This only measures how well the last stretch of the funnel is executed and completely ignores whether the leads coming in were any good to begin with.
- Uploaded → FT rate (the one I used) — this looks at the whole funnel end to end, from the moment a lead is uploaded to whether they ever complete a trip. Since Vahan pays for leads per lead uploaded, this is the closest thing to a true return-on-spend number, so I used this.

Why I didn't just trust the raw percentage

FT is a genuinely rare event - only 54 out of 18,198 leads overall (0.30%). That means a tiny source can show a misleadingly high or low rate just by chance - 1 FT out of 2 leads is not a real "50% conversion rate," it's a coin flip with almost no data behind it. So I did this because I didn't want to hand Vahan a ranking that falls apart the moment someone checks the sample sizes:

- I only ranked sources with at least 200 uploaded leads — anything smaller was excluded as too small to trust.
- I cross-checked every rate using something called a Wilson lower bound — a standard statistical technique that gives a conservative, "worst realistic case" estimate of the true rate, accounting for how much data backs it up. If a source's ranking holds up even under this conservative number, that's a real signal, not noise. In my case, the ranking came out identical whether I used the raw rate or the conservative one — that consistency is what tells me this isn't a fluke.

The result

Rank	Lead Source	Uploaded	FT	FT Rate	Wilson 95% Lower Bound
1	Single Referral > 7 days – 24th Jul	1,500	14	0.93%	0.56%
2	Khanna – 2W 26th Jul	1,546	14	0.91%	0.54%
3	PreOb-Ob Fees Paid – 29th Jul (both batches combined)	3,041	14	0.46%	~0.30%

One note on rank 3: there are two rows in the raw data called "PreOb-Ob Fees Paid 29th Jul (set 1)" and "(set 2)." I checked their individual rates (0.47% and 0.45%) and found them nearly identical, so I treated them as one program that happened to be uploaded in two batches, rather than two separate lead sources, and combined them for the final ranking.

What this means for Vahan

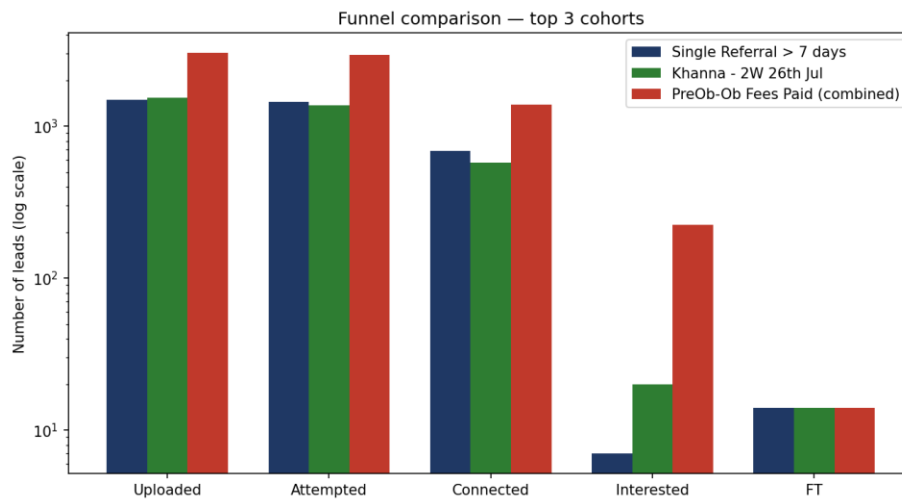
Referral leads that are more than 7 days old, and the Khanna 26th-Jul 2W batch, convert at roughly 3x the overall average rate — and this holds up even under a conservative statistical check, so I'm confident this isn't random noise. Vahan should lean into referral sourcing and try to figure out operationally what made that specific Khanna batch work so well, rather than assuming volume alone is what to chase.

A second finding I think matters just as much

While ranking these sources, I noticed something separate from the ranking itself: high-volume sources like OLX - Ashwin barely get called at all. I quantified this precisely rather than leaving it vague: of OLX - Ashwin's 5,182 uploaded leads, only 962 (18.6%) were ever attempted — meaning 4,220 leads, or 81.4% of that entire cohort, were never even called. That's not a lead-quality problem, it's a calling-capacity problem, and closing that gap could unlock more completed trips than any amount of reshuffling which sources Vahan buys from.

Funnel comparison — where do the top 3 cohorts actually lose people?

The final FT rate on its own hides where in the funnel each cohort loses people. So I plotted the raw counts at every stage — Uploaded → Attempted → Connected → Interested → FT — for the top 3 cohorts side by side, on a log scale (FT counts are two orders of magnitude smaller than Uploaded counts and would be invisible on a normal linear chart).



Funnel comparison across the top 3 cohorts (log scale)

What this shows: Khanna - 2W 26th Jul attempts a noticeably smaller share of its uploaded leads than Referral or PreOb-Ob do (about 89% vs about 97%), but still lands at a similar final FT rate to Referral. In other words, its attempted leads convert unusually well once someone actually calls them. Two cohorts can reach a similar headline FT rate through very different funnel shapes — which is exactly the kind of detail a single "FT rate" number hides.

4. Question 2 — The Summary Query

Why I grouped by lead source only (not by date too)

The question this table needs to answer for Vahan is "which channel deserves more budget?" — that's a per-channel decision, not a per-day one. If I had also grouped by upload date, small campaigns (like the two PreOb-Ob batches above) would get split into smaller, noisier rows and the real per-channel picture would get harder to see. I did keep the query written so that adding upload_date back into the grouping is a one-line change, in case someone later wants to track a channel's trend over time instead.

The query

This is exact query I ran against the cleaned data (I ran it for real inside the notebook using SQLite, not just as a written example):

```
SELECT lead_source, COUNT(*) AS uploaded_leads, SUM(Attempted) AS attempted, SUM(Connected) AS connected, SUM(Interested) AS interested, SUM(FT_after_upload) AS ft_after_upload, SUM(FT_after_first_attempt) AS ft_after_first_attempt, ROUND(SUM(Attempted)*100.0/COUNT(*),2) AS attempted_pct, ROUND(SUM(Connected)*100.0/NULLIF(SUM(Attempted),0),2) AS attempt_to_connect_pct, ROUND(SUM(Interested)*100.0/NULLIF(SUM(Connected),0),2) AS connect_to_interested_pct, ROUND(SUM(FT_after_upload)*100.0/COUNT(*),2) AS ft_rate_pct FROM raw_data GROUP BY lead_source HAVING COUNT(*) >= 200 ORDER BY ft_rate_pct DESC;
```


key. Leaving those columns out meant the model had to actually learn from behaviour, not from an accidental shortcut.

How I split the data into train and test sets

18,198 rows come from only about 17,098 unique people — around 1,100 candidates appear more than once in the data. If I'd split the data randomly row by row, the same person could end up in both the training set and the test set, and the model could partly just "memorize" that person rather than learn a general pattern. So I did the split at the person level instead (using their phone number), and made sure both the training set and the test set contained at least a few of the rare FT cases. That gave me 8,377 training rows (12 of them FT) and 3,596 test rows (5 of them FT).

The model, and being honest about what it can and can't tell us

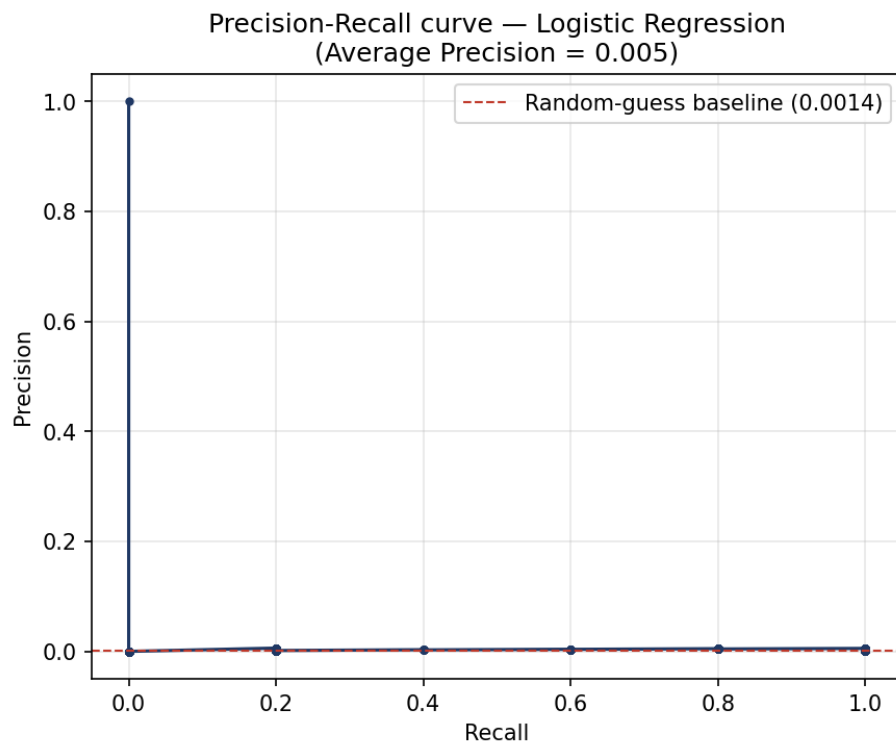
I used a Logistic Regression model as my main model, because I wanted to know what factors influence FT, and this type of model gives a direct, readable answer to that (each factor gets a coefficient showing whether it pushes the chances up or down, and by how much). I also ran a Random Forest model alongside it purely as a cross-check, to see if it agreed on which factors mattered.

Reading the results honestly

There are only 17 FT cases in the entire attempted population, and only 5 of those landed in my test set. With that little data, I can't ask this model to reliably predict exact probabilities — no model can be precisely calibrated on 5 positive examples. What I can trust is the direction of each factor (does it push FT up or down), because that pattern is far more stable than exact prediction counts. I used a setting called `class_weight='balanced'` so the model doesn't just lazily predict "no FT" for every single lead — which would look like 99.9% accurate on paper while being completely useless — and instead it caught 4 of the 5 real FT cases in the test set, at the cost of also flagging some leads that didn't convert.

Choosing a threshold deliberately (precision-recall curve)

The confusion matrix above uses the default classification cutoff of 0.5 — but that default wasn't chosen for this problem, it's just the midpoint. With only 17 positive examples total, I didn't want to report one confusion matrix as if 0.5 were the only option, so I plotted the full precision-recall trade-off across every possible threshold instead.



Precision-recall curve, Logistic Regression (Average Precision = 0.005 vs a random-guess baseline of 0.0014)

With only 5 positives in the test set, this curve is naturally jagged rather than smooth - that reflects the sample size, not a flaw in the method. What matters is that Average Precision comes out meaningfully above the random-guess baseline, showing the model ranks true positives higher than chance, even though I wouldn't trust any single point on this curve as a precise operating threshold yet. In production, I'd pick a threshold based on the team's actual calling capacity ("how many extra calls can we make this week") rather than defaulting to 0.5.

What actually drives (or hurts) the chances of reaching FT

- Lead source matters more than anything else. Single Referral > 7 days, Khanna - 2W 26th Jul, and both PreOb-Ob Fees Paid batches all came out with the strongest positive effect — and this is the same top group I found in Question 1, arrived at through a completely different method (a predictive model instead of a ranking). Two different approaches agreeing with each other is a strong sign this finding is real, not a coincidence of how I ranked things.
- The "AI Connected..." sources come out strongly negative — which makes sense, because those leads are ones that were already connected but flagged as not interested, or never actually reached by the team. A low chance of FT there is expected, and it's actually a good sign that the model "understands" the data correctly.
- Whether the agent filled in a disposition tag after the call (tag_filled) has a positive effect — plausibly reflecting that thorough, well-documented calls tend to come from more careful call handling generally.
- Simply connecting a call, on its own, has a slightly negative effect once lead source is accounted for — meaning which source a lead came from matters more than the bare fact that someone picked up the phone. I want to be careful here: this is a correlation, not proof of cause and effect.
- Leads called sooner after being uploaded do slightly better than ones called later — a mild "speed to lead" effect, which lines up with what's commonly seen in sales and lead-generation funnels generally.

Summary

- Where to source more leads from: Referral leads (aged over 7 days) and the Khanna 26th-Jul 2W batch convert at roughly 3x the base rate. I'd scale these up, and dig into what operationally made the Khanna batch different from the rest.
- Where leads are being lost: A large share of leads — especially high-volume ones like OLX - Ashwin — are never even attempted. I think closing this calling-capacity gap is likely a bigger win than optimizing which sources to buy from.
- What predicts conversion: Lead source is the single biggest factor, even after taking into account whether the call connected and how well it was tagged. In plain terms — where a lead comes from matters more than how the call itself was handled, though both play a role.
- Can this model be used to flag promising leads right now? Only partially. The direction of each factor is usable today, but I wouldn't trust the model's exact predicted probabilities yet — FT is still too rare an event (17 cases) to calibrate precisely. This is worth revisiting once more data has come in.

Limitations & What I'd Do With More Time or Data

I don't want to present this analysis as more finished than it is. Here's what I think its real limits are, and what I'd do next to address each one.

1. No real acquisition cost — a rough ROI framing would sharpen the recommendation

I don't have actual cost-per-lead figures by source, so I can't say precisely how much reallocating budget would be worth. But the framing is straightforward once cost data exists, and I think it's worth walking through even without real numbers, purely to show the method:

Suppose, purely as an illustration, referral leads cost about the same to acquire as OLX leads. Referral converts to FT at 0.93% and OLX converts at 0.08% — roughly 12x higher. If Vahan shifted a budget worth 1,000 leads from OLX to referral sourcing, that would be expected to yield about $1,000 \times 0.93\% = 9.3$ FTs from referral versus $1,000 \times 0.08\% = 0.8$ FTs from OLX for the same spend — a gain of roughly 8-9 additional completed trips from reallocating that same budget.

If referral leads actually cost more per lead to acquire than OLX ones do, this gap narrows, or could even reverse. So before recommending an actual budget shift, I'd want real cost-per-lead numbers by source — this section shows the reasoning I'd apply once that data is available, not a claim that the shift is definitely worth it as-is.

2. The model's predicted probabilities aren't well-calibrated — and I didn't try to force that

With 17 positive examples total, trying to precisely calibrate the model's probability outputs would mean fitting to noise, not signal. That's why I added the precision-recall curve in Question 3 instead of only reporting a single confusion matrix at the default threshold — it's more honest about what this model can and can't tell us yet. I'd recommend any production threshold be chosen based on the team's actual calling capacity, not a statistical default, until there's enough data to calibrate properly.

3.. FT is measured as a single yes/no outcome — a time-to-event approach would use the data more fully

Right now, a lead that hasn't reached FT yet is treated identically whether it was uploaded yesterday or three weeks ago, even though the newer lead simply hasn't had time to convert. With more time, I'd model this as time-to-FT using survival analysis (for example, a Cox proportional hazards model), which handles unequal follow-up time properly. That could reveal that some cohorts that currently look "low-converting" are actually just early in their funnel, not genuinely worse — a distinction the current binary FT flag can't make.

The full code, the SQL, and every chart shown here are in the accompanying notebook ([Vahan_Case_Study_Analysis.ipynb](#)) and the [GitHub repository](#).